Nasazení webu - výběr technologie pro provoz, nasazení webu do provozu

---

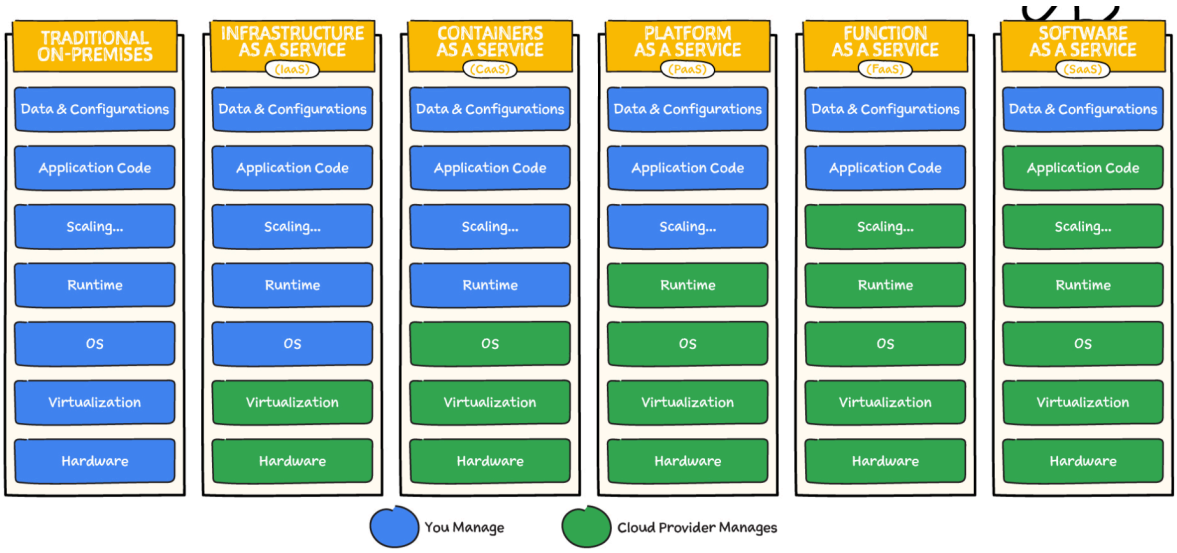
Nasazení webu

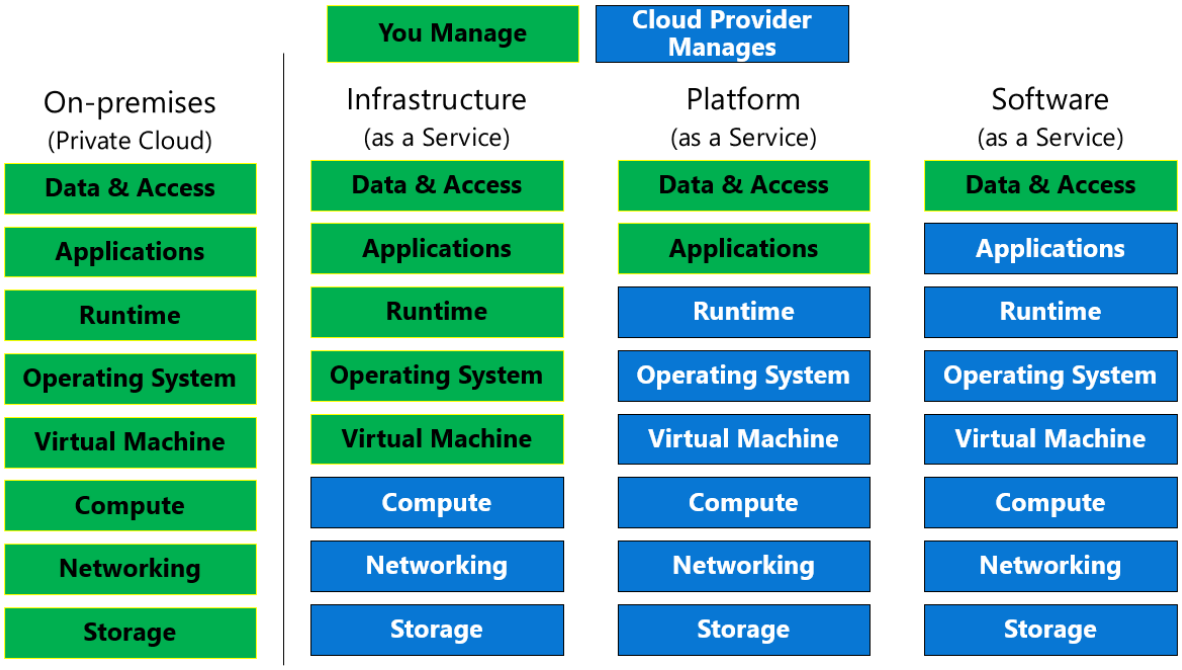
- Proces, při kterém přeneseme hotovou webovou aplikaci z vývojového prostředí do produkčního, aby byla dostupná koncovým uživatelům přes internet

---

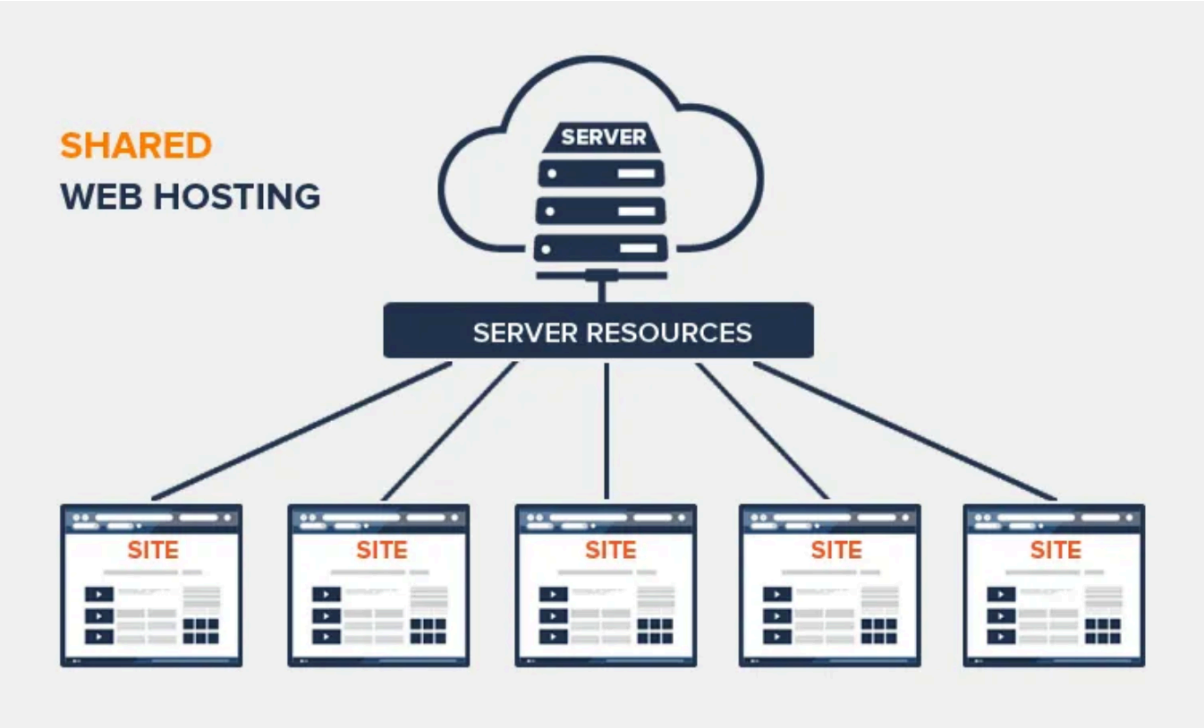
Výběr technologie pro provoz

- **cloudové služby:**
  - **IaaS (Infrastrucure as a Service):**
    - Poskytovatel pronajímá výpočetní výkon svých strojů zákazníkům
    - Například v podobě VPS, cloud hostingu nebo dedikovaného serveru
    - Jenom si pronajímáme fungující hardware o zbytek se staráme sami
  - **PaaS (Platform as a Service):**
    - Poskytovatel se stará o server, operační systém a aktualizace
    - Vývojáři jen vytvoří aplikaci a nahrají svůj kód bez toho, aby stavěli infrastrukturu nebo platformu po svém
    - Nevýhodou může být vysoká závislost na poskytovateli ohledně výběru software, který lze na platformě nainstalovat
  - **SaaS (Software as a Service):**
    - Hotové řešení
    - Poskytuje celé cloud-based řešení, kteří zákazníci mohou používat
    - Produkty **SaaS** jsou celkově spravované poskytovatelem
    - Fungující softwarový systém, který nemusíme spravovat
  - Jaké řešení si chceme vybrat vybíráme podle toho kolik kontroly a odpovědnosti chceme nad danou aplikací mít
- **On-premises:**
  - Řešení, kde spravujeme vše
  - Všechno vlastníme od hardwaru až po samotnou aplikaci

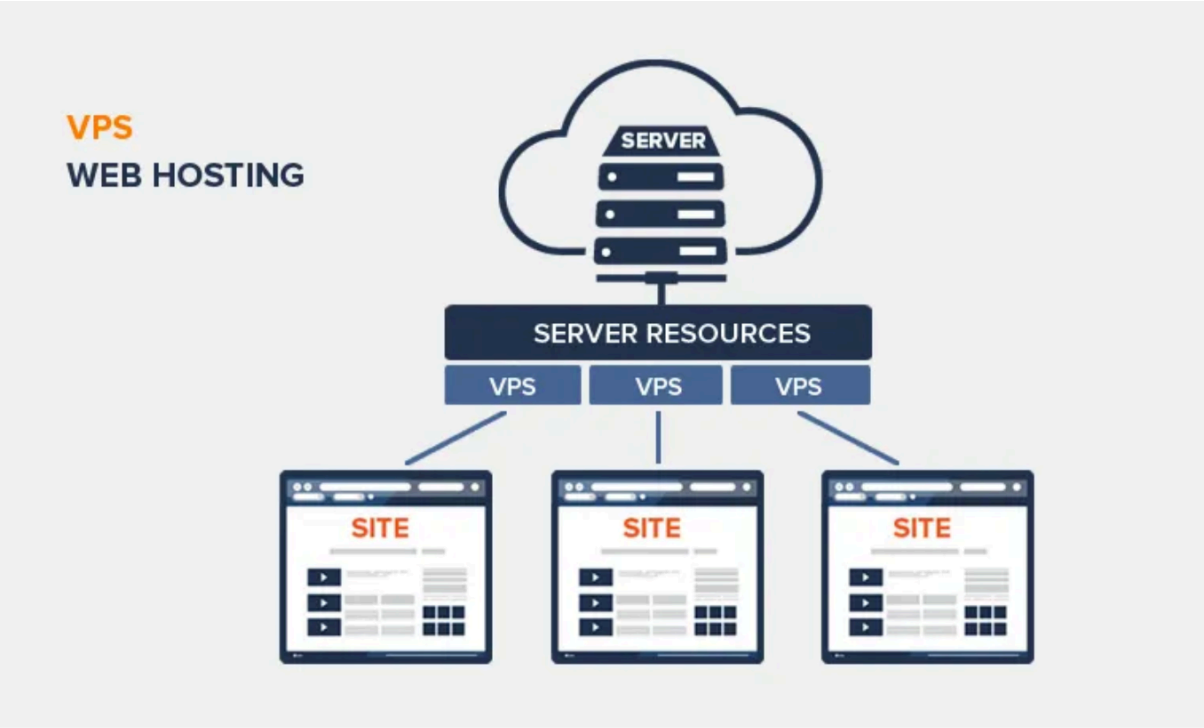

TRADITIONAL ON-PREMISES
INFRASTRUCTURE AS A SERVICE (IaaS)
CONTAINERS AS A SERVICE (CaaS)
PLATFORM AS A SERVICE (PaaS)
FUNCTION AS A SERVICE (FaaS)
SOFTWARE AS A SERVICE (SaaS)
Data & Configurations
Application Code
Scaling...
Runtime
OS
Virtualization
Hardware
You Manage
Cloud Provider Manages


You Manage
Cloud Provider Manages
On-premises (Private Cloud)
Infrastructure (as a Service)
Platform (as a Service)
Software (as a Service)
Data & Access
Applications
Runtime
Operating System
Virtual Machine
Compute
Networking
Storage

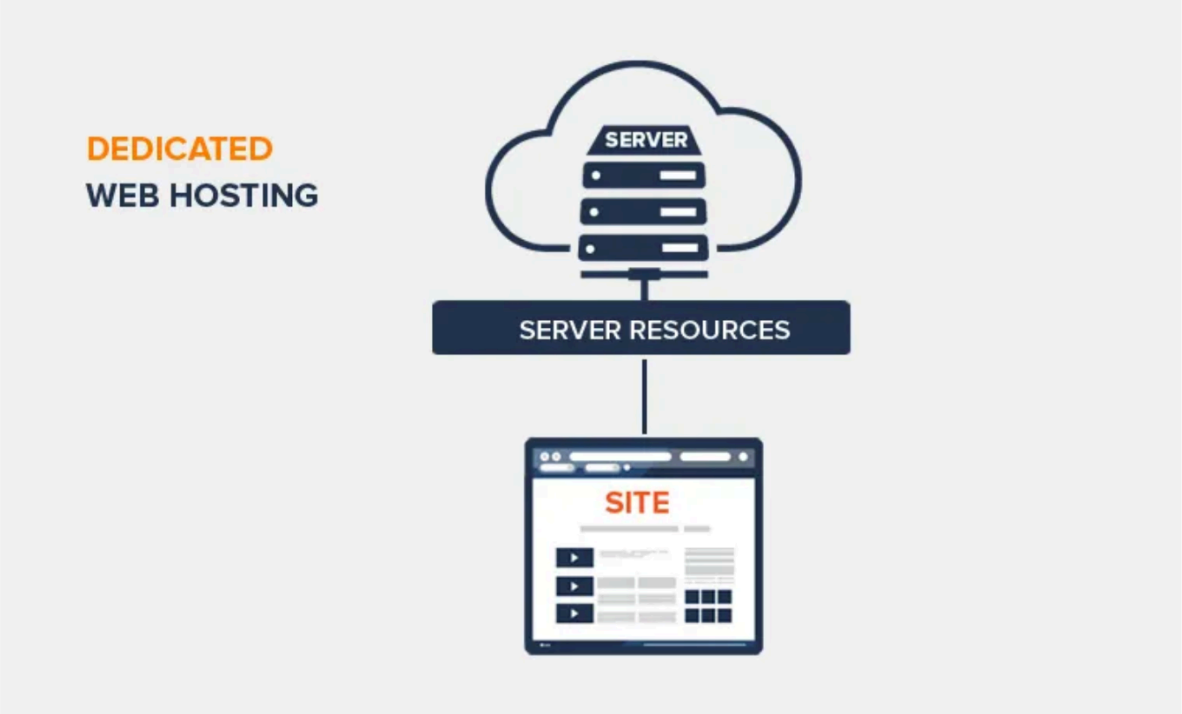
- **Konkrétní typy hostingu:**
  - **Sdílený hosting:**
    - Náš web běží na jednom serveru společně s desítkami až stovkami dalších webů
    - Všichni sdílí stejné prostředky (procesor, RAM, diskový prostor
    - Nemáme kontrolu nad konfigurací serveru, nemůžeme instalovat vlastní software ani měnit nastavení
    - **Výhody:**
      - Levné
      - Lehké na nastavení
    - **Nevýhody:**
      - Méně kontroly
      - Možné problémy s výkonem



- **VPS (virtual private server):**
  - Simuluje vlastní dedikovaný stroj
  - Na jednom fyzickém serveru běží více VPS, ale každé ma vyhrazené prostředky -> určitý počet jader CPU, RAM a velikosti disku
  - Můžeme si nainstalovat co potřebujeme -> libovolný operační systém, webový server, databázi, programovací jazyk
  - Ostatní weby na stejném fyzickém stroji neovlivňují náš výkon
  - Spadá pod model **IaaS**
  - **Výhoda**/**Nevýhoda**:
    - O vše se staráme my -> umět nastavit server, řešit aktualizace, zabezpečení, zálohy

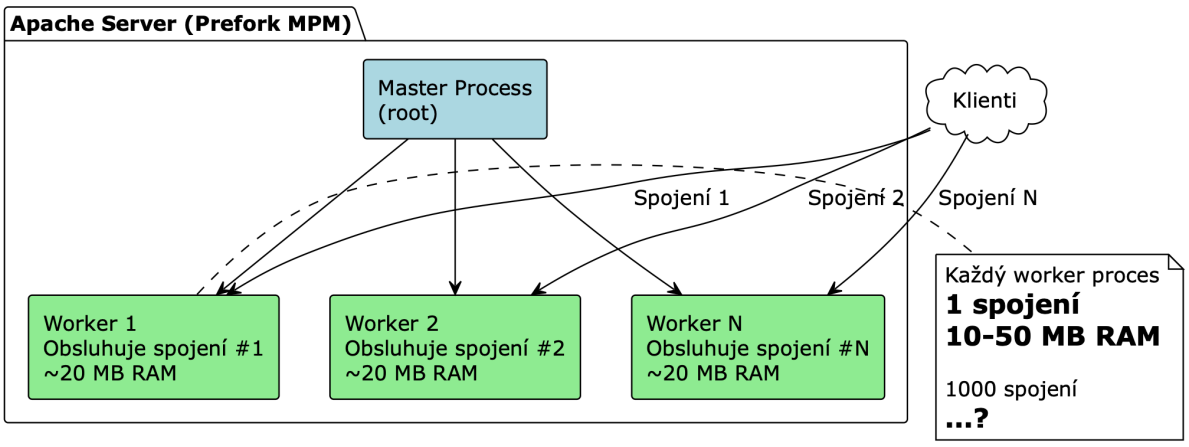


- **Dedikovaný server:**
  - Celý fyzický stroj vyhrazený jen pro nás
  - Žádné sdílený, veškerý výkon je náš
  - **Výhody:**
    - Dobrý výkon
    - Celková kontrola
  - **Nevýhody:**
    - Vysoká cena
    - Technické znalosti



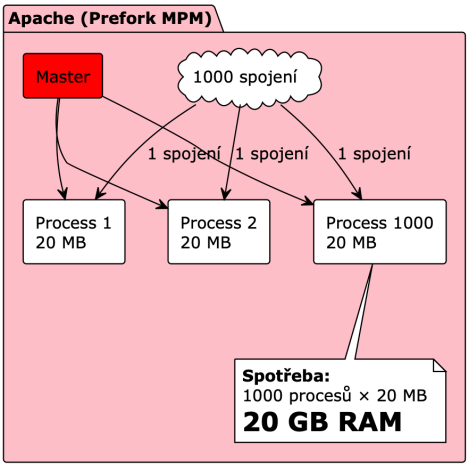
Výběr webového serveru

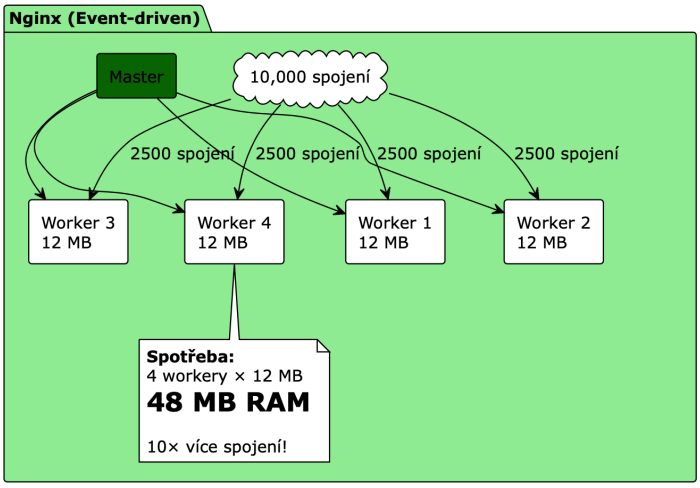
- **Apache**
  - Vytvořen v roce 1995
  - Nejpopulárnější webserver světa (1996-2019)
  - Open source
  - Použití dnes:
    - WordPress hosting, sdílený hosting, legacy aplikace, PHP s mod_php
  - Používá Multi-Processing Modules (MPM)
  - **Procesový model -> Prefork:**
    - **Prefork MPM (tradiční):**
      - Jeden proces per spojení
      - Žádná vlákna - maximální stabilita
      - Vysoká spotřeba paměti
  - **Procesový model -> Worker:**
    - **Worker MPM (hybrid):**
      - Kombinace procesů a vláken
      - Nižší spotřeba paměti než **Prefork**
      - Vyžaduje thread-safe knihovny
  - **Procesový model -> Event:**
    - **Event MPM (moderní):**
      - Lepší pro keep-alive spojení
      - Asynchronní zpracování některých operací
      - Doporučení pro Apache 2.4+



- **Co je .htaccess:**
  - Konfigurace Apache per adresář
  - Umožňuje uživatelům měnit nastavení bez přístupu k hlavní konfiguraci
- **Kdy použít?**
  - **Výhody:**
    - Snadná konfigurace a setup
    - **.httaccess** pro sdílený hosting
    - Dobrý pro tradiční LAMP stack
    - Kompatibilita se starými aplikacemi
  - **Nevýhody**:
    - Vysoká spotřeba paměti při mnoha spojeních
    - Pomalejší než **Nginx**
    - Špatně školuje při tisících souběžných spojení
    - Složitá konfigurace pro moderní use cases

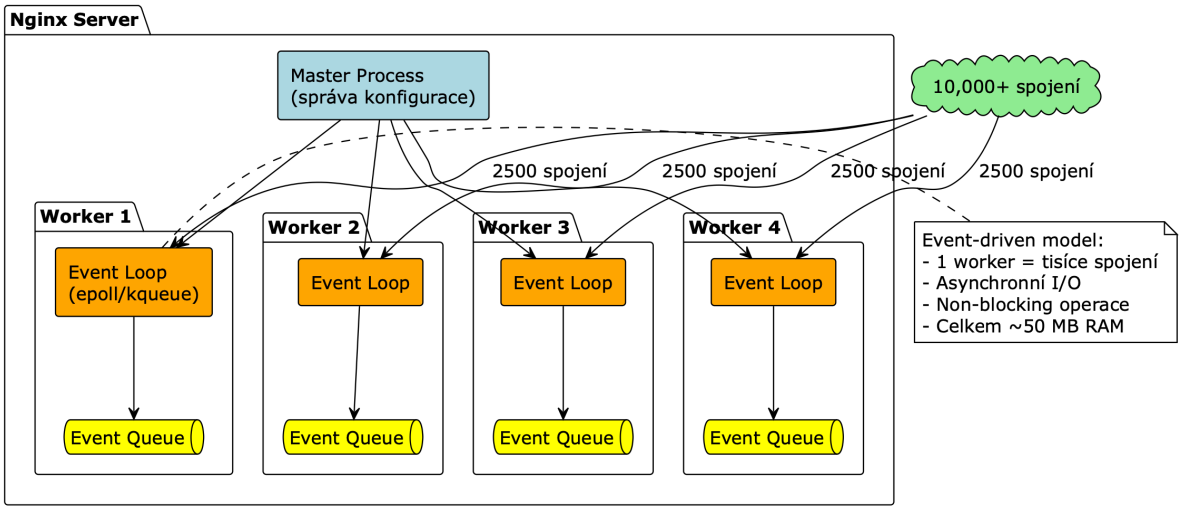
- **Nginx (2004):**
  - Řeší C10K problem, který má **Apache**
  - Event-driven, asynchronní architektura
  - Jeden worker process = tisíce spojení
  - Nízká spotřeba paměti
  - Vysoký výkon





/

- **Architektura:**
  - Event-driven, asynchronní, non-blocking
  - **<u>Master process:</u>**
    - Správa konfigurace
    - Spawn worker procesů
    - Reload bez downtime
  - **<u>Worker processes:</u>**
    - Zpracování požadavků
    - Event loop (epoll/kqueue)
    - Jeden worker = tisíce spojení



- **Event-driven model:**
  - Jak funguje event loop:
    1. Worker čeká na událost
    2. Když přijde, worker ji rychle zpracuje
    3. Pokud operace blokuje, worker pokračuje v jiné práci
    4. Až je operace hotová, worker se k ní vrátí

- **Výhody:**
  - Žádné čekání na I/O - asynchronní operace
  - Minimální context switching
  - Nízká spotřeba paměti (žádné procesy per spojení)
  - Skvělý výkon při tisících souběžných spojení
- **Kdy použít:**
  - **Výhody:**
    - Vysoký výkon při tisících souběžných spojení
    - Nízká spotřeba paměti
    - Výborný pro statické soubory
    - Ideální jako reverse proxy / load balancer
    - Moderní architektura
  - **Nevýhody:**
    - Složitější konfigurace než Apache
    - Žádná podpora .htaccess
    - Změny vyžadují reload
    - Méně modulů třetích stran
    - Strmější křivka učení
- Srovnání Apache vs Nginx:

Metrika	Apache	Nginx
Statické soubory	Pomalejší	Rychlejší (2–3x)
Dynamický obsah	Podobný	Podobný
Memory per connection	10–50 MB	1–2 MB
Max. connections	Stovky–tisíce	Desítky tisíc
CPU usage	Vyšší	Nižší

- **Use case - kdy použít Apache:**
  - **Sdílený hosting**
    - Uživatelé potřebují .htaccess pro vlastní konfiguraci
  - **WordPress a PHP aplikace**
    - Mnoho pluginů a témat spoléhá na .htaccess
    - mod_php pro jednoduchost
  - **Legacy aplikace**
    - Starší kód napsaný pro Apache
    - Potřeba specifických Apache modulů
  - **Týmy znalé Apache**
    - Existující know-how
    - Není nutnost přechodu
- **Use case - kdy použít Nginx:**
  - **High-traffic weby**
    - Tisíce souběžných spojení
    - Potřeba školovat s omezenými zdroji
  - **Statické soubory a CDN**
    - Obrázky, videa, CSS, JavaScript
    - Maximální výkon při servírování statiky
  - **Reverse proxy / Load balancer**
    - Distribuce zátěže mezi backend servery
    - API gateway pro mikroslužby
  - **Moderní aplikace**
    - Node.js, Python, Go backends
    - Kontejnerové prostředí

- **Moderní alternativy:**
  - **Caddy:**
    - Moderní webserver s automatickým HTTPS
    - Automatické HTTPS - Let's Encrypt built-in
    - Jednoduchá konfigurace - Caddyfile
    - Reverse proxy, load balancing
    - Napsáno v Go
    - **Výhody:**
      - Nejjednodušší konfigurace
      - Automatické SSL
      - Moderní protokoly
      - zero-downtime reload
    - **Nevýhody:**
      - Méně mature než Nginx
      - Menší komunita
      - Méně pluginů
  - **Traefik:**
    - cloud-native reverse proxy
    - Auto-discovery služeb (Docker, Kubernetes)
    - Dynamická konfigurace - žádné reloady
    - Automatický Let's Encrypt
    - Load balancing, circuit breakers
    - Napsáno v Go
    - Ideální pro kontejnerové prostředí, kde služby přicházejí a odcházejí dynamicky

Server	Kdy použít
Caddy	Nejjednodušší setup, automatické HTTPS bez konfigurace, malé až střední weby
Traefik	Mikroslužby, dynamická konfigurace
Nginx	High-traffic, produkční nasazení, maximum výkonu a stability
Apache	Legacy aplikace, WordPress na sdíleném hostingu, potřeba .htaccess

Dev vs production servery:

- **Development server:**
  - Hot Module Replacement (HMR)
  - Automatický reload
  - Debugging tools
  - Volná bezpečnost
  - Source maps
  - Není optimalizovaný
- **Production server:**
  - Optimalizovaný výkon
  - Bezpečnostní hardening
  - Komprese, caching
  - SSL/TLS
  - Rate limiting
  - Monitoring a logy
- **Vite dev server:**
  - Vite -> moderní development server
  - Hot Module Replacement (HMR) -> okamžitý refresh bez reload stránky
  - Matice ES modules - prohlížeč načítá moduly přímo
  - Rychlý start - žádný bundling při startu
  - Optimalizováno pro moderní frontend frameworks
  - **Proč jen pro development?:**
    - **<u>Dev server:</u>**
      - Soubory servírovány on-the-fly, HMR pro rychlý vývoj, source maps pro debugging, žádná optimalizace
    - **<u>Production build:</u>**
      - vite build vytvoří optimalizované bundle, minifikace, tree-shaking, code splitting, statické soubory pro Nginx/Apache

HTTPS - Let’s Encrypt

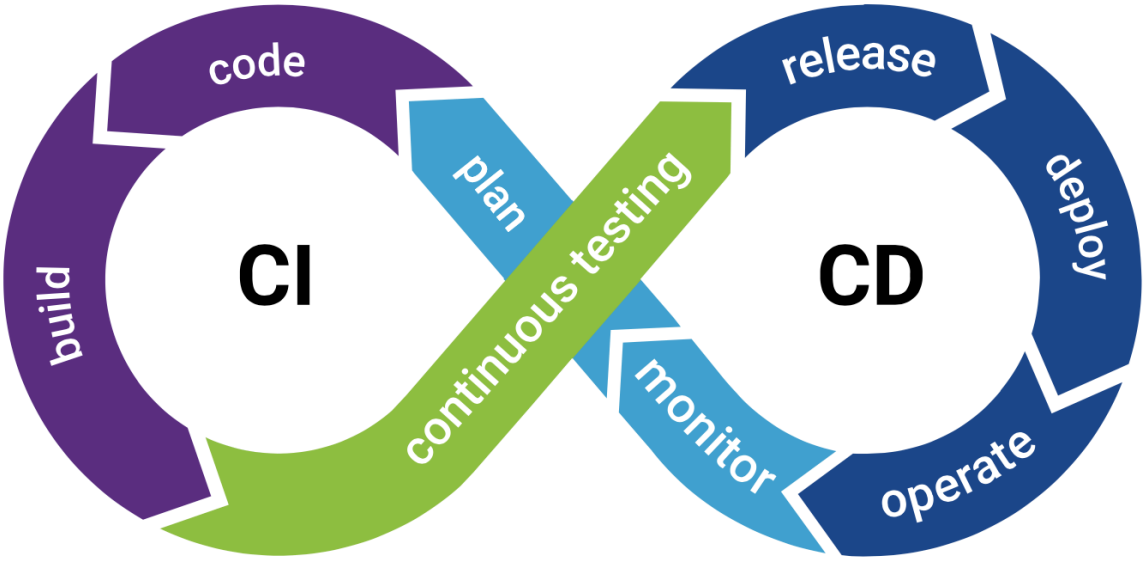
- Zdarma SSL certifikáty
- Automatizované vystavení certifikátů (Certbot)
- 90 dní platnosti - automatická obnova
- Domain Validation (DV) - ověření vlastnictví domény
- Podporováno všemi moderními prohlížeči
- Vody použít HTTPS v produkci !!!!! HTTP je nezabezpečené

Doména a DNS

- Místo IP adresy potřebujeme něco co bude čitelný pro uživatele
- Takže si potřebujeme koupit doménu
- **DNS:**
  - Překládá doménové jméno na IP adresu
  - **DNS záznamy:**
    - A = propojuje doménu s IPv4 adresou serveru, nejzákladnější záznam
    - AAAA = totéž, ale platí pro IPv6 adresy

Proces nasazení - jak kód doručíme na server

- **Ruční nasazení:**
  - Nejzákladnější způsob
  - Připojíme se k serveru a nahrajeme soubory ručně např. přes SSH
  - **Funguje, ale má nevýhody**
    - Je pomalé
    - Náchylné k lidským chybám
    - Nemáme automatizované testy a neexistuje snadná cesta zpět
- **Nasazení přes Git:**
  - Standart
  - Kód máme na verzovacím systému Git na platformě např. GitHub
  - Na serveru spustíme git pull a stáhneme aktuální verzi kódu
  - **Výhody:**
    - Kompletní historie změn
    - Možnost vrátit se k jakékoli předchozí verzi
    - Více lidí může pracovat na stejném projektu
- **CI/CD pipeline (Continuous Integration / Continuous Deployment):**
  - **CI:**
    - Znamená, že při každé změně, který developer udělal a commitnul, tak se automaticky spustí testy a sestavení aplikace
    - Pomáhá detekovat bugy, nějaké jiné potíže brzo při vývoji
  - **CD:**
    - Pokud vše projde, aplikace se automaticky nasadí do produkce
  - **Výhody:**
    - Rychlejší, kvalitní kód
  - Celý tento proces jde nastavit například na Github prostřednictvím Github Actions



- **Docker:**
  - Aplikace zabalíme do kontejneru, který obsahuje vše potřebné
  - **Dockerfile** popisuje, jak se kontejner sestaví
  - Často se kombinuje s CI/CD pipeline